Electrothon: Gate-Go-Brr

Secure Access Terminal — Embedded & IoT

Project Report — Phase 2: Cloud Sync

Platform	ESP32 (Wokwi simulation)
Backend	Node.js + Express + SQLite
Wokwi Project Link	https://wokwi.com/projects/466788091244715009

1. Project Overview

This project implements a two-phase secure access terminal on an ESP32. Phase 1 provides fully local, network-independent access control: a 4x4 keypad, an I2C LCD, RGB LED and buzzer feedback, role-based access control (RBAC) via hardcoded C++ structs, a non-blocking input timeout, and a security lockout after repeated failed attempts. Phase 2 extends this into a connected system: the terminal syncs to real time over NTP, serializes every access event into JSON, and pushes it to a remote server — gracefully queuing events locally whenever the network is unavailable and automatically flushing that queue once connectivity returns.

Beyond the two required phases, the implementation also adds two extension features: admin-triggered runtime member enrollment (new IDs can be added from the keypad itself, without reflashing) and a check-in / check-out toggle, where a second successful entry for the same ID logs that person out rather than granting access again — turning the terminal into a lightweight attendance system in addition to an access gate.

2. Code Architecture and State Machine Logic

The firmware is built entirely around one global **state** variable and a single **loop()** function that behaves as a state machine, never a long-running procedural script. Every screen the terminal can show — idle, typing, granted, denied, locked out, or any step of enrollment — is a distinct enum value, and the entire loop() body is a sequence of "if the state is X, do Y" checks. This is the single design decision that keeps the whole system non-blocking: instead of the code pausing with delay() calls, every timed behaviour (input timeout, lockout duration, feedback display time, enrollment timeout) is implemented by storing a millis() timestamp when entering a state and comparing elapsed time against it on every subsequent loop() tick.

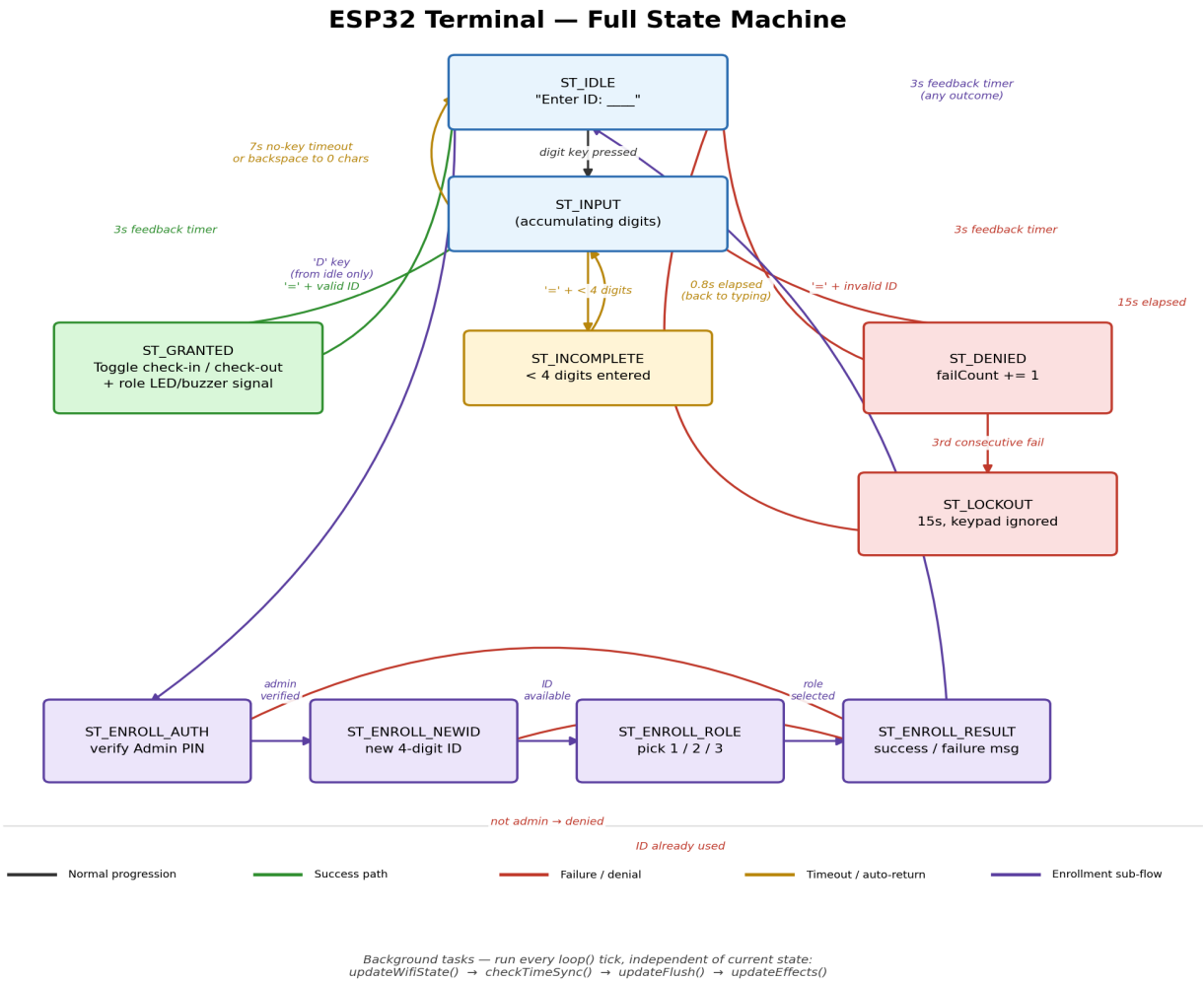

Figure 1: Full terminal state machine. Edge color indicates outcome type (success, failure, timeout, or enrollment); see legend on the diagram.

2.1 Non-blocking LED/buzzer feedback

Role-specific LED and buzzer feedback (e.g. Admin's triple-blink-then-solid-green pattern) was originally implemented with delay() between each digitalWrite()/tone() call. This was refactored into a table-driven effect scheduler: each pattern is expressed as an array of steps (EffectStep { ledOnPin, ledOffPin, toneFreq, toneDur, hold }), and a single updateEffects() function — called on every loop() tick alongside the other background tasks — advances one step whenever enough time has elapsed. This removed all delay() calls from the codebase while keeping the exact same visual/audio timing as the original blocking version.

2.2 Background tasks

Four tasks run unconditionally at the top of every loop() iteration, regardless of what state the terminal is in: WiFi status polling (throttled to once every 2s), NTP sync detection, offline-queue flush attempts (throttled to once every 5s), and the effect scheduler tick. This separation means the network layer and the local access-control layer never block each other.

3. Offline Queue Design

Every access event — check-in, check-out, denied attempt, lockout trigger, or enrollment — is serialized into a small JSON object and handed to sendOrQueue(). This function is the single decision point in the whole networking layer:

- If WiFi is connected **and** NTP time has synced, it attempts an immediate HTTP POST to the server.
- If either precondition fails, or the POST attempt itself fails (timeout, server unreachable, etc.), the event is pushed onto a fixed-size array (offlineQueue[10]) instead of being lost.

The queue is a plain, statically-sized String array — not a dynamically growing list — capped at 10 entries as required. When a new event arrives and the queue is already full, the oldest entry is discarded to make room (a FIFO "drop-oldest" policy): every element is shifted left by one index, and the new payload is written into the final slot. This choice was made deliberately so the most recent activity is always preserved even if the terminal spends an extended period offline, at the cost of losing the very earliest queued events in that scenario.

Recovery is handled by updateFlush(), which runs at most once every 5 seconds while WiFi is up and the queue is non-empty. It walks the queue from front to back, POSTing each entry in order and stopping at the very first failure — this preserves chronological order and ensures a mid-flush disconnect never causes entries to be sent out of sequence or silently dropped. Once a stretch of leading entries is confirmed delivered, the array is compacted (the sent entries shifted out) and the count decremented, so the queue only ever holds what genuinely still needs to be sent.

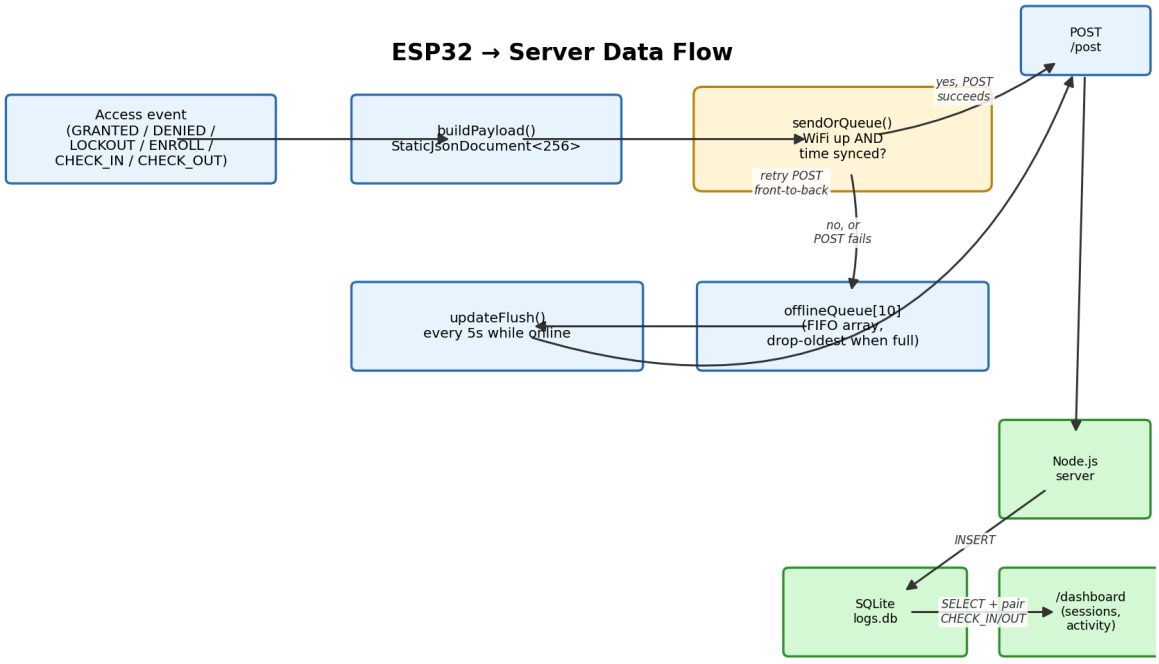

Figure 2: End-to-end path of a single access event, from generation on the ESP32 through to the dashboard.

4. Memory Management and Avoiding Heap Fragmentation

All JSON construction uses ArduinoJson's **StaticJsonDocument<256>** rather than DynamicJsonDocument. StaticJsonDocument allocates its entire working buffer on the stack at compile time with a fixed capacity, so

building a payload never touches the heap at all — there is no malloc/free cycle per event, and therefore no opportunity for heap fragmentation to accumulate over the terminal's runtime, however many access events it logs.

256 bytes was chosen as a safety margin above the actual payload size: fields are five short key/value pairs at most (timestamp, result, id, name, role), which ArduinoJson's own capacity calculator estimates comfortably under 200 bytes even with the longest possible strings (a 16-byte name field plus a 20-character ISO timestamp). Sizing generously but fixed avoids the two failure modes of a mis-sized static buffer: overflow (if too small, which ArduinoJson would simply fail to serialize with no warning) and unnecessary stack pressure (if far larger than needed).

The one place dynamic memory is unavoidable is the Arduino String class, used for the queue entries and HTTP payload bodies. This is a deliberate, contained trade-off rather than an oversight: the queue is capped at exactly 10 String objects at all times (never grows past that), each holding a short serialized JSON string, so the total live String memory is bounded and predictable rather than open-ended.

5. Extension Features

5.1 Runtime Member Enrollment

USERS[] was converted from a fixed 4-entry const array of string-literal pointers into a mutable, 10-slot array of fixed character buffers (MAX_USERS = 10), allowing new members to be written into unused slots at runtime. Enrollment is gated behind an existing Admin ID (verified via the same findUser() lookup used for login) and guards against duplicate IDs and a full member table before committing a new entry.

5.2 Check-In / Check-Out Toggle

Each User carries a checkedIn boolean. A successful login toggles this flag: false→true is logged and displayed as a check-in (welcome message, standard success signal); true→false is logged and displayed as a check-out (personalized goodbye message, a distinct LED/buzzer pattern). The server pairs CHECK_IN/CHECK_OUT events per member ID to compute session durations and to surface who is currently still checked in, on the /dashboard page.

6. Constraints Compliance

- **No delay():** confirmed absent from the final source — every timed behaviour uses millis()-based comparisons instead.
- **Only Wokwi-standard components:** 4x4 keypad, I2C LCD, discrete LEDs, buzzer — no non-simulated hardware.
- **Modular architecture:** distinct functions per concern (screens, signals, network state, queue, JSON building) rather than one monolithic loop().

7. Known Limitations

- Enrolled members and check-in state live only in ESP32 RAM; a reboot resets the member table to the 4 built-in users and clears all check-in flags (though server-side history is unaffected).

- HTTPS requests use setInsecure() (no certificate validation) — acceptable for a demo/hackathon server, not suitable for production without a proper certificate chain.
- The offline queue's drop-oldest policy means an extended offline period (more than 10 events) will lose the earliest events from that period.